



SH320 Data Engineering

Spring 2026 Semester

Pandas

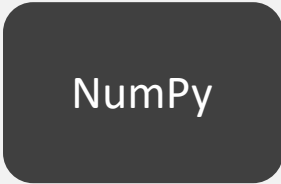
5 Data Manipulation: Part 2

Hyun Ahn

School of Computing and Artificial Intelligence

hyunahn@hs.ac.kr

Core Libraries for Data Science

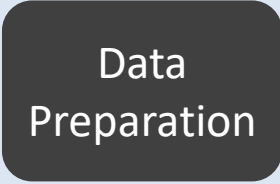
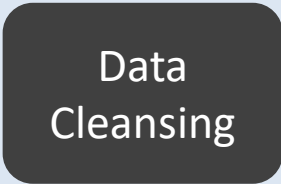
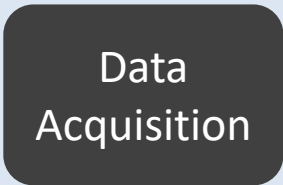


- **[W2] N-Dimensional Arrays**
 - NumPy arrays
 - Array manipulation
- **[W3] Efficient Computation in NumPy**
 - Universal functions
 - Advanced features



- **[W5] Data Manipulation: Part 1**
 - Pandas objects: Series, DataFrame
 - Implicit / explicit indexing
- **[W6] Data Manipulation: Part 2**
 - Pandas objects & manipulation
 - Combining datasets
 - Exploratory data analysis

Data Engineering



5.1 데이터셋 결합

Combining Data



- Pandas 에서는 서로 다른 데이터 소스를 결합하기 위한 메서드들을 제공
 - `concat()`: 여러 개의 DataFrame(또는 Series)을 단순 연결
 - `merge()`, `join()`: 두 개의 DataFrame을 공통 열 또는 인덱스를 기준으로 병합

Recall: Concatenation of NumPy Arrays



- NumPy의 `concatenate()`: 두 개 이상의 배열(또는 리스트)를 연결

```
x = [1, 2, 3]
y = [4, 5, 6]
z = [7, 8, 9]

np.concatenate([x, y, z])

array([1, 2, 3, 4, 5, 6, 7, 8, 9])
```

Concatenate Pandas Objects



- Pandas의 `concat()`: `np.concatenate()` 보다 다양한 연결 옵션 제공
- 함수 시그니처

`pandas.concat(objs, axis=0, join='outer', ignore_index=False, keys=None, levels=None, names=None, verify_integrity=False, sort=False, copy=True)` [\[source\]](#)

- `objs`: 연결대상 객체들
- `axis`: 연결 차원 축
- `join`: 조인 옵션
- `ignore_index`: 기존의 인덱스 무시

Simple Concatenation with concat()



- 예제: Series 객체 연결

```
ser1 = pd.Series(['A', 'B', 'C'], index=[1, 2, 3])  
ser2 = pd.Series(['D', 'E', 'F'], index=[4, 5, 6])
```

```
pd.concat([ser1, ser2])
```

```
1    A  
2    B  
3    C  
4    D  
5    E  
6    F  
dtype: object
```

concat(): DataFrame Objects



- DataFrame 생성 함수 정의

```
def make_df(cols, ind):  
    data = {c: [str(c) + str(i) for i in ind]  
            for c in cols}  
  
    return pd.DataFrame(data, ind)
```

```
make_df('ABC', [1, 2])
```

	A	B	C
1	A1	B1	C1
2	A2	B2	C2

- DataFrame 객체 연결: 행(세로) 방향으로 연결

```
df1 = make_df('AB', [1, 2])
```

df1

	['A']	['B']
[1]	A1	B1
[2]	A2	B2

```
df2 = make_df('AB', [3, 4])
```

df2

	['A']	['B']
[3]	A3	B3
[4]	A4	B4

```
pd.concat([df1, df2])
```

	['A']	['B']
[1]	A1	B1
[2]	A2	B2
[3]	A3	B3
[4]	A4	B4

- 열(가로) 방향으로 연결: `axis = 1`

```
df3 = make_df('AB', [0, 1])
```

	['A']	['B']
[0]	A0	B0
[1]	A1	B1

```
df4 = make_df('CD', [0, 1])
```

	['C']	['D']
[0]	C0	D0
[1]	C1	D1

```
pd.concat([df3, df4], axis=1)
```

	['A']	['B']	['C']	['D']
[0]	A0	B0	C0	D0
[1]	A1	B1	C1	D1

np.concatenate() vs. pd.concat()



- concatenate() 보다 concat() 이 더 편한 이유
 - 인덱스 중복 허용
 - 인덱스 중복 에러 처리: `verify_integrity=True`
 - 인덱스 무시: `ignore_index=True`
 - 계층적 인덱싱 지원: `keys=[keys]` ([참고](#))

Concatenation with Joins

- concat() 조인 옵션: 서로 다른 열 집합을 가지는 객체를 연결

x	['A']	['B']	['C']
[1]	A1	B1	C1
[2]	A2	B2	C2

y	['B']	['C']	['D']
[3]	B3	C3	D3
[4]	B4	C4	D4

pd.concat([x, y], join='outer')

pd.concat([x, y], join='inner')

	['A']	['B']	['C']	['D']
[1]	A1	B1	C1	NaN
[2]	A2	B2	C2	NaN
[3]	NaN	B3	C3	D3
[4]	NaN	B4	C4	D4

	['B']	['C']
[1]	B1	C1
[2]	B2	C2
[3]	B3	C3
[4]	B4	C4

join='outer'
외부조인 = 합집합

join='inner'
내부조인 = 교집합

concat의 조인 연산은 열(column) 집합에 대해 수행되며, 행 집합은 기본적으로 합집합으로 계산합니다.

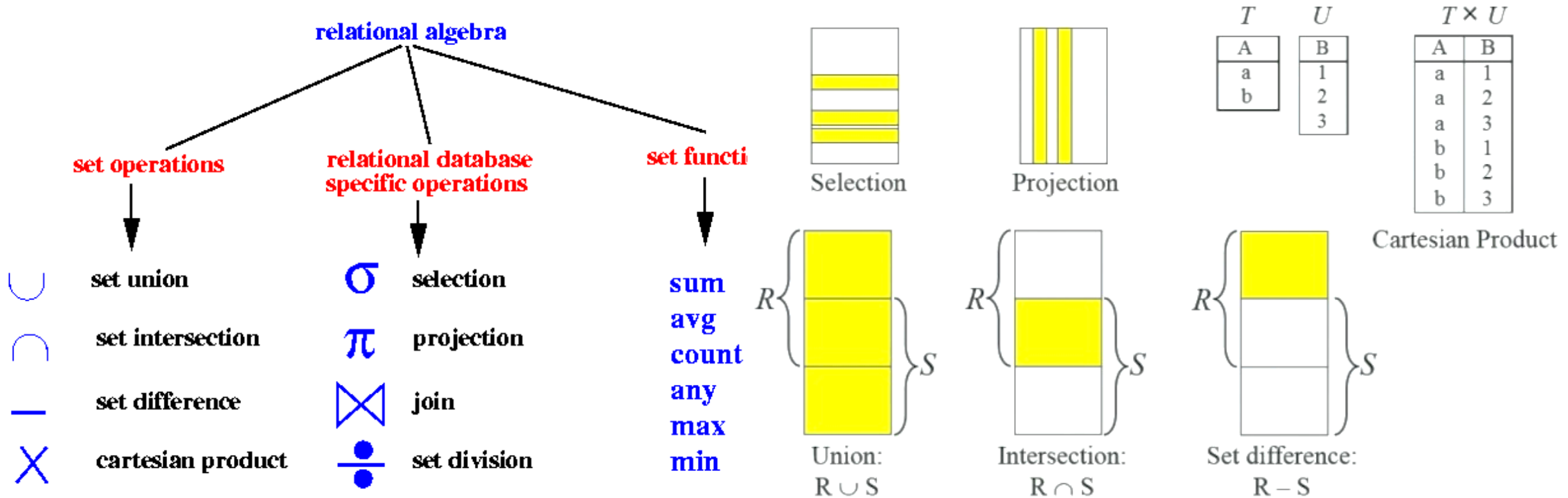
Merge and Join



- Pandas 는 고성능 인메모리 조인과 병합 연산 기능을 제공
- `merge()`: 데이터셋 조인/병합 관련 **다양한 옵션** 제공
- `join()`: **인덱스**를 기준으로 두 DataFrame 을 결합

Relational Algebra

- 관계 대수: 관계형 데이터베이스의 연산들을 수학적으로 기술



Source: Medium ([link](#))

Source: Connolly, T., Database Systems

Relational Algebra ex)



Person

Name	Age	Weight
Harry	34	80
Sally	28	64
George	29	70
Helena	54	54
Peter	34	80

관계대수 연산:

$$\rightarrow \sigma_{name, age}(\pi_{age \geq 34}(R1)) \rightarrow$$

SQL 표현:

```
SELECT name, age  
FROM person  
WHERE age >= 34;
```

Name	Age
Harry	34
Helena	54
Peter	34

pd.merge() 에는 관계 대수의 일부분이 구현되어 있음

Categories of Joins



- **조인 연산 유형**

- 일대일 One-to-One **조인**
- 다대일 Many-to-One **조인**
- 다대다 Many-to-Many **조인**

⇒ 공통 열의 항목 중복 여부에 따라 결정됨

One-to-One Join

- 일대일 조인: 공통 열의 중복 항목 없음

- 공통 열(예: employee)을 키로 인식하고 이를 중심으로 조인을 수행

df1 ['employee'] ['group']

[0]	Bob	Accounting
[1]	Jake	Engineering
[2]	Lisa	Engineering
[3]	Sue	HR

df2 ['employee'] ['hire_date']

[0]	Lisa	2004
[1]	Bob	2008
[2]	Jake	2012
[3]	Sue	2014

df3 = pd.merge(df1, df2)

df3 ['employee'] ['group'] ['hire_date']

[0]	Bob	Accounting	2008
[1]	Jake	Engineering	2012
[2]	Lisa	Engineering	2004
[3]	Sue	HR	2014

Many-to-One Join

- 다대일 조인: 두 DataFrame 의 키 중에 하나가 중복된 항목을 포함
 - 조인 결과에 중복된 항목을 포함

df3 ['employee'] ['group'] ['hire_date']

[0]	Bob	Accounting	2008
[1]	Jake	Engineering	2012
[2]	Lisa	Engineering	2004
[3]	Sue	HR	2014

df4 ['group'] ['supervisor']

[0]	Accounting	Carly
[1]	Engineering	Guido
[2]	HR	Steve

pd.merge(df3, df4)

['employee'] ['group'] ['hire_date'] ['supervisor']

[0]	Bob	Accounting	2008	Carly
[1]	Jake	Engineering	2012	Guido
[2]	Lisa	Engineering	2004	Guido
[3]	Sue	HR	2014	Steve

Many-to-Many Join



- 다대다 조인: 두 DataFrame 의 키가 모두 중복된 항목을 포함

df1 ['employee'] ['group']

[0]	Bob	Accounting
[1]	Jake	Engineering
[2]	Lisa	Engineering
[3]	Sue	HR

pd.merge(df1, df5)

df5 ['group'] ['skills']

[0]	Accounting	math
[1]	Accounting	spreadsheets
[2]	Engineering	coding
[3]	Engineering	linux
[4]	HR	spreadsheets
[5]	HR	organization

	['employee']	['group']	['skills']
[0]	Bob	Accounting	math
[1]	Bob	Accounting	spreadsheets
[2]	Jake	Engineering	coding
[3]	Jake	Engineering	linux
[4]	Lisa	Engineering	coding
[5]	Lisa	Engineering	linux
[6]	Sue	HR	spreadsheets
[7]	Sue	HR	organization

Specifying the Merge Key



- `merge()` 에서 제공하는 데이터 병합 옵션들
 - `on`: 병합에 사용할 공통 키
 - `left_on`, `right_on`: 공통 키 역할을 하는 두 DataFrame 의 열
 - `left_index`, `right_index`: 인덱스를 이용한 병합 여부

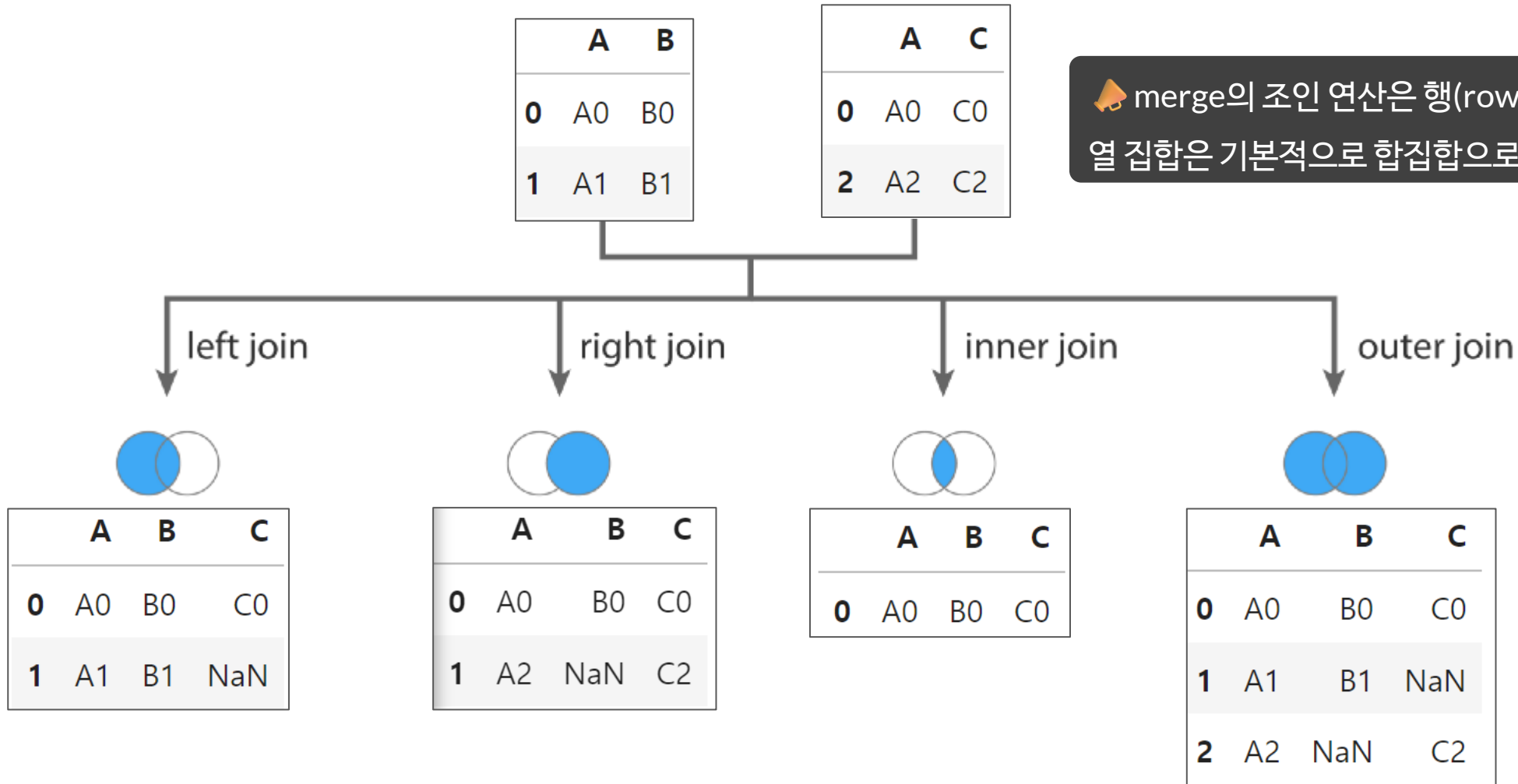
Specifying Set Arithmetic for Joins



- `merge()` 조인 옵션
 - `how=inner`: 내부 조인(디폴트) $\Rightarrow$ 교집합
 - `pd.concat()`: 외부 조인이 디폴트
 - `how=outer`: 외부 조인 $\Rightarrow$ 합집합
 - `how=left`: 왼쪽 조인 (첫 번째 DataFrame 기준으로 조인)
 - `how=right`: 오른쪽 조인 (두 번째 DataFrame 기준으로 조인)

Join Options: Left / Right / Inner / Outer

🔔 merge의 조인 연산은 행(row) 집합에 대해 수행되며, 열 집합은 기본적으로 합집합으로 계산합니다.



5.2 탐색적 데이터 분석

Exploratory Data Analysis



- **탐색적 데이터 분석** EDA: Exploratory Data Analysis
 - 데이터를 본격적으로 모델링하거나 분석하기 전에 데이터를 이해하는 과정
- **주요 목적**
 - 데이터는 어떤 구조를 가지고 있나? ⇨ 데이터 요약(기술통계 등)
 - 이상치는 있는가? ⇨ 박스플롯 시각화
 - 변수들 간의 관계는 어떤가? ⇨ 산점도 시각화
 - 데이터 분포는 어떤 모양인가? ⇨ 히스토그램 시각화
 - 결측치는 있는가? ⇨ 결측값 확인 및 처리

🔔 EDA는 단순한 데이터 살펴보기가 아니라,
분석 방향을 결정하는 핵심 단계입니다.

• RISE Regional Innovation System & Education **사업**

- 대학을 지역사회의 중심축 역할을 하여 지역 발전과 혁신을 유도하기 위한 대학 지원사업
- 25년 사업 선정: 경기대+한신대+협성대 컨소시엄으로 5년간 50억 수혜
 - 한신대의 주요 역할: AI+SW 융합 및 AI 기반 제조 교육

• 본 수업과의 연관성

- 경기도 지역 기업 전문가 특강 또는 보고서 평가 예정

06	『Chap. 6: Pandas』 <6.1 Pandas 객체 및 조작> - Pandas 주요 객체 및 속성 - Pandas 객체 생성 및 조작 <6.2 데이터셋 결합> - Pandas 객체 연결 - DataFrame 조인 연산	
07	『Chap. 7: Data Acquisition』 <7.1 웹스크래핑> - 웹스크래핑 개요 - 웹스크래핑 실습 <7.2 웹서비스> - RESTful API - RESTful 기반의 데이터 수집	
08	- 중간 시험 대체 프로그램 과제 - 지역특화산업(부리기술) 및 GX 기업 전문가 특강	- 전반기 학습 내용을 종합하여 수행하는 데이터 분석 과제
09	『Chap. 8: Working with Time Series』 <8.1 시계열 데이터 생성> - 시계열 객체 생성 - 정규 시퀀스 - 빈도 및 오프셋 <8.2 시계열 전용 연산> - 리샘플링 - 시프팅 - 윈도우	

DX vs AX



- **DX** Digital Transformation, 디지털 전환
 - 아날로그 방식의 업무 프로세스, 제품, 서비스를 **디지털 기술**(Cloud, IoT, Big Data 등)을 **활용해 디지털로 전환**하는 것
 - 핵심: ‘**데이터화**’와 ‘**연결**’. 종이 문서를 없애고(Paperless), 공장의 설비를 인터넷에 연결하여 현황을 실시간으로 모니터링
- **AX** AI Transformation, 인공지능 전환
 - DX로 구축된 디지털 환경과 데이터 위에 **AI 기술**을 입혀, 기업의 체질을 ‘**지능형**’으로 근본적으로 바꾸는 것
 - 핵심: ‘**자율화**’와 ‘**최적화**’. 사람이 데이터를 보고 판단하던 것을 AI가 학습하여 스스로 판단하고, 예측하며, 결과물을 생성

- **KAMP**Korea AI Manufacturing Platform

- 중소벤처기업부에서 주관하는 민관협력 제조특화 AI 플랫폼 ([link](#))
- 제조데이터 수집·저장·분석 인프라, AI 전문가, 실증 서비스 등을 한 곳에 모아, 중소 제조기업이 제조 AI 를 효율적으로 활용할 수 있도록 통합 지원

- 49종의 무료 제조AI데이터셋 제공



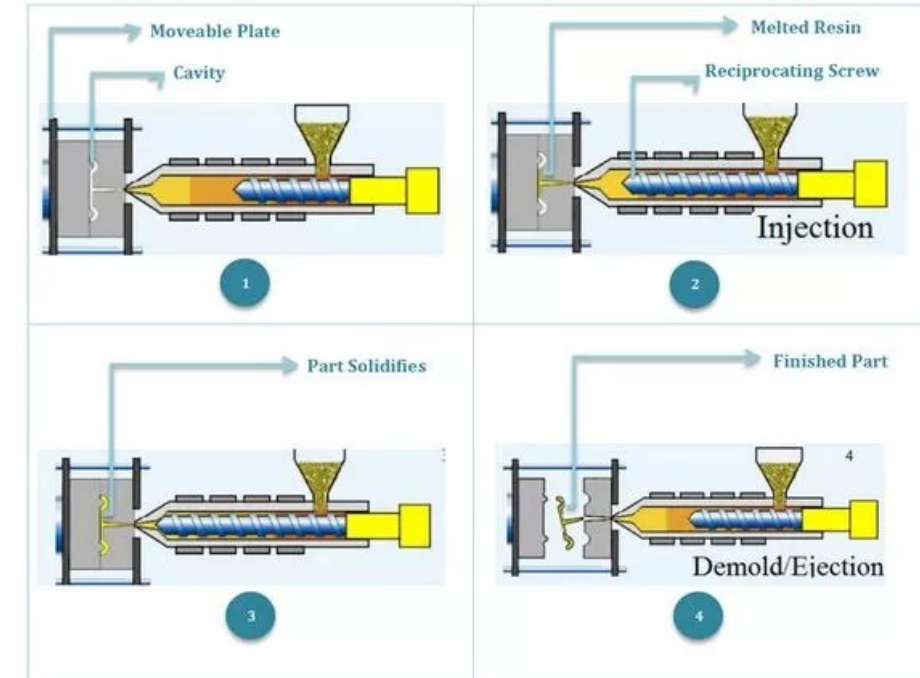
KAMP: Injection Molding Machine

• 데이터셋 개요

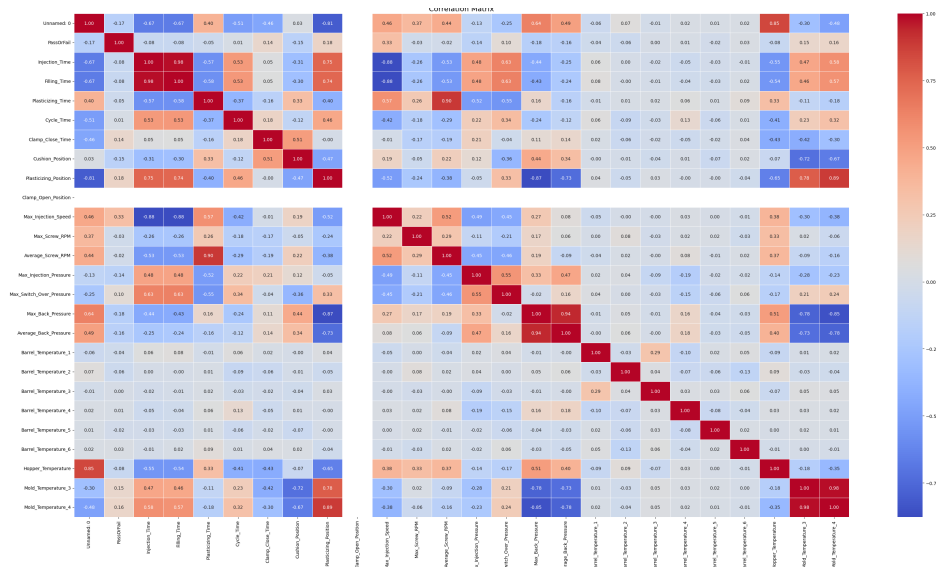
- 출처: 사출성형기 AI 데이터셋, KAMP ([link](#))
- 제조 공정명: 자동차 앞유리 사이드 몰딩사출 공정 ([link](#))
- 수집 기간: 2020.10 ~ 2020.11 (약 1개월)
- 데이터 건수: 약 8,000건 (Cycle 단위로 집계된 데이터)

• 문제 정의

- 목표: 불량품이 발생하지 않는 '최적의 설비 세팅값' 찾기
- 주요 변수(X): 유압 온도, 충전 압력, 충전 시간, 사출 속도 등
- 목표 변수(Y): 불량 여부(양품 / 불량)



- 두 변수가 함께 변하는 정도와 방향을 나타내는 통계적 척도로, 한 변수의 변화가 다른 변수의 변화와 얼마나 관련되어 있는지를 수량화한 것
- 가설: “플라스틱을 녹이는 거니까 '온도'가 가장 중요하지 않을까?”
- Heatmap 분석 결과: 불량률(Y)과 용융 온도(X)의 상관계수는 0.2 이하로 상관성이 낮음
- 인사이트: 단순히 온도만 높다고 불량률이 나는 게 아니라, 여러 조건이 복합적으로 작용함



Unnamed: 0	1.00	-0.17	-0.67	-0.67	0.40	-0.51	-0.46	0.03	-0.81
	-0.17	1.00	-0.08	-0.08	-0.05	0.01	0.14	-0.15	0.18
	-0.67	-0.08	1.00	0.98	-0.57	0.53	0.05	-0.31	0.75
	-0.67	-0.08	0.98	1.00	-0.58	0.53	0.05	-0.30	0.74
	0.40	-0.05	-0.57	-0.58	1.00	-0.37	-0.16	0.33	-0.40
	-0.51	0.01	0.53	0.53	-0.37	1.00	0.18	-0.12	0.46
	-0.46	0.14	0.05	0.05	-0.16	0.18	1.00	0.51	-0.00
	0.03	-0.15	-0.31	-0.30	0.33	-0.12	0.51	1.00	-0.47
	-0.81	0.18	0.75	0.74	-0.40	0.46	-0.00	-0.47	1.00

Clamp_Open_Position									
Max_Injection_Speed	0.46	0.33	-0.88	-0.88	0.57	-0.42	-0.01	0.19	-0.52
Max_Screw_RPM	0.37	-0.03	-0.26	-0.26	0.26	-0.18	-0.17	-0.05	-0.24
Average_Screw_RPM	0.44	-0.02	-0.53	-0.53	0.90	-0.29	-0.19	0.22	-0.38
Max_Injection_Pressure	-0.13	-0.14	0.48	0.48	-0.52	0.22	0.21	0.12	-0.05
Max_Switch_Over_Pressure	-0.25	0.10	0.63	0.63	-0.55	0.34	-0.04	-0.36	0.33
Max_Back_Pressure	0.64	-0.18	-0.44	-0.43	0.16	-0.24	0.11	0.44	-0.87
Average_Back_Pressure	0.49	-0.16	-0.25	-0.24	-0.16	-0.12	0.14	0.34	-0.73
Barrel_Temperature_1	-0.06	-0.04	0.06	0.08	-0.01	0.06	0.02	-0.00	0.04
Barrel_Temperature_2	0.07	-0.06	0.00	-0.00	0.01	-0.09	-0.06	-0.01	-0.05
Barrel_Temperature_3	-0.01	0.00	-0.02	-0.01	0.02	-0.03	-0.02	-0.04	0.03
Barrel_Temperature_4	0.02	0.01	-0.05	-0.04	0.06	0.13	-0.05	0.01	-0.00
Barrel_Temperature_5	0.01	-0.02	-0.03	-0.03	0.01	-0.06	-0.02	-0.07	-0.00
Barrel_Temperature_6	0.02	0.03	-0.01	0.02	0.09	0.01	0.04	0.02	-0.04
Hopper_Temperature	0.85	-0.08	-0.55	-0.54	0.33	-0.41	-0.43	-0.07	-0.65
Mold_Temperature_3	-0.30	0.15	0.47	0.46	-0.11	0.23	-0.42	-0.72	0.78
Mold_Temperature_4	-0.48	0.16	0.58	0.57	-0.18	0.32	-0.30	-0.67	0.89

0.46	0.37	0.44	-0.13	-0.25	0.64	0.49	-0.06	0.07	-0.01	0.02	0.01	0.02	0.85	-0.30	-0.48
0.33	-0.03	-0.02	-0.14	0.10	-0.18	-0.16	-0.04	-0.06	0.00	0.01	-0.02	0.03	-0.08	0.15	0.16
-0.88	-0.26	-0.53	0.48	0.63	-0.44	-0.25	0.06	0.00	-0.02	-0.05	-0.03	-0.01	-0.55	0.47	0.58
-0.88	-0.26	-0.53	0.48	0.63	-0.43	-0.24	0.08	-0.00	-0.01	-0.04	-0.03	0.02	-0.54	0.46	0.57
0.57	0.26	0.90	-0.52	-0.55	0.16	-0.16	-0.01	0.01	0.02	0.06	0.01	0.09	0.33	-0.11	-0.18
-0.42	-0.18	-0.29	0.22	0.34	-0.24	-0.12	0.06	-0.09	-0.03	0.13	-0.06	0.01	-0.41	0.23	0.32
-0.01	-0.17	-0.19	0.21	-0.04	0.11	0.14	0.02	-0.06	-0.02	-0.05	-0.02	0.04	-0.43	-0.42	-0.30
0.19	-0.05	0.22	0.12	-0.36	0.44	0.34	-0.00	-0.01	-0.04	0.01	-0.07	0.02	-0.07	-0.72	-0.67
-0.52	-0.24	-0.38	-0.05	0.33	-0.87	-0.73	0.04	-0.05	0.03	-0.00	-0.00	-0.04	-0.65	0.78	0.89

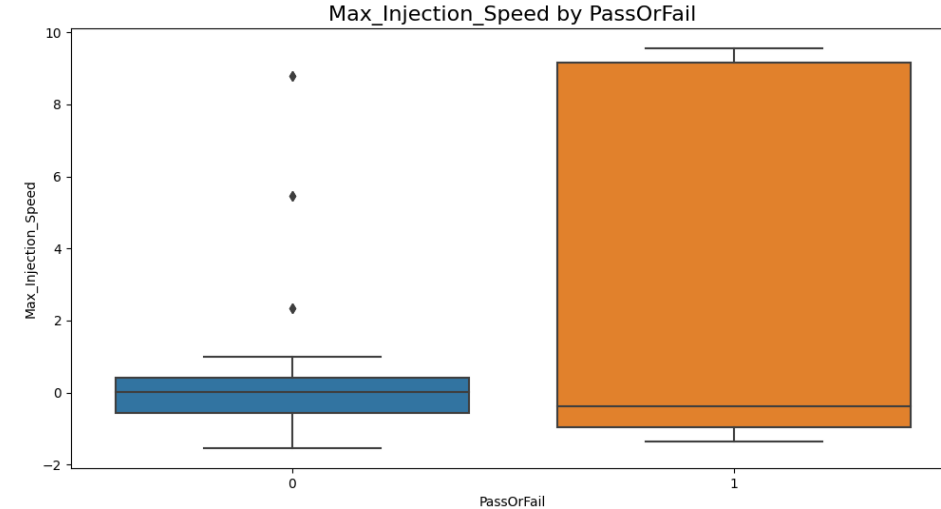
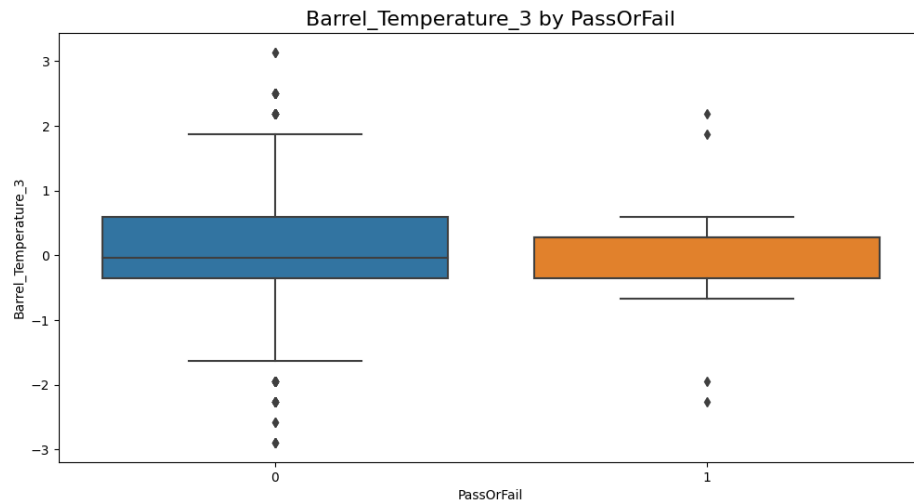
1.00	0.22	0.52	-0.49	-0.45	0.27	0.08	-0.05	-0.00	-0.00	0.03	0.00	-0.01	0.38	-0.30	-0.38
0.22	1.00	0.29	-0.11	-0.21	0.17	0.06	0.00	0.08	-0.03	0.02	0.02	-0.03	0.33	0.02	-0.06
0.52	0.29	1.00	-0.45	-0.46	0.19	-0.09	-0.04	0.02	-0.00	0.08	-0.01	0.02	0.37	-0.09	-0.16
-0.49	-0.11	-0.45	1.00	0.55	0.33	0.47	0.02	0.04	-0.09	-0.19	-0.02	-0.02	-0.14	-0.28	-0.23
-0.45	-0.21	-0.46	0.55	1.00	-0.02	0.16	0.04	0.00	-0.03	-0.15	-0.06	0.06	-0.17	0.21	0.24
0.27	0.17	0.19	0.33	-0.02	1.00	0.94	-0.01	0.05	-0.01	0.16	-0.04	-0.03	0.51	-0.78	-0.85
0.08	0.06	-0.09	0.47	0.16	0.94	1.00	-0.00	0.06	-0.00	0.18	-0.03	-0.05	0.40	-0.73	-0.78
-0.05	0.00	-0.04	0.02	0.04	-0.01	-0.00	1.00	-0.03	0.29	-0.10	0.02	0.05	-0.09	0.01	0.02
-0.00	0.08	0.02	0.04	0.00	0.05	0.06	-0.03	1.00	0.04	-0.07	-0.06	-0.13	0.09	-0.03	-0.04
-0.00	-0.03	-0.00	-0.09	-0.03	-0.01	-0.00	0.29	0.04	1.00	0.03	0.03	0.06	-0.07	0.05	0.05
0.03	0.02	0.08	-0.19	-0.15	0.16	0.18	-0.10	-0.07	0.03	1.00	-0.08	-0.04	0.03	0.03	0.02
0.00	0.02	-0.01	-0.02	-0.06	-0.04	-0.03	0.02	-0.06	0.03	-0.08	1.00	0.02	0.00	0.02	0.01
-0.01	-0.03	0.02	-0.02	0.06	-0.03	-0.05	0.05	-0.13	0.06	-0.04	0.02	1.00	-0.01	0.00	-0.01
0.38	0.33	0.37	-0.14	-0.17	0.51	0.40	-0.09	0.09	-0.07	0.03	0.00	-0.01	1.00	-0.18	-0.35
-0.30	0.02	-0.09	-0.28	0.21	-0.78	-0.73	0.01	-0.03	0.05	0.03	0.02	0.00	-0.18	1.00	0.98
-0.38	-0.06	-0.16	-0.23	0.24	-0.85	-0.78	0.02	-0.04	0.05	0.02	0.01	-0.01	-0.35	0.98	1.00



EDA: Distributions

- 분포: “불량은 어디에 숨어있을까?”

- 박스플롯 시각화: 양품일 때와 불량일 때의 변수 분포 비교
- 발견: ‘최대 사출 속도’에서 양품/불량 집단의 차이가 명확하게 갈림
- 가설 수정: 온도는 기본적으로 제어되고 있으니, 변동성이 큰 사출 속도가 핵심 원인일 것이다



EDA: Feature Importance



- **변수 중요도** Feature Importance

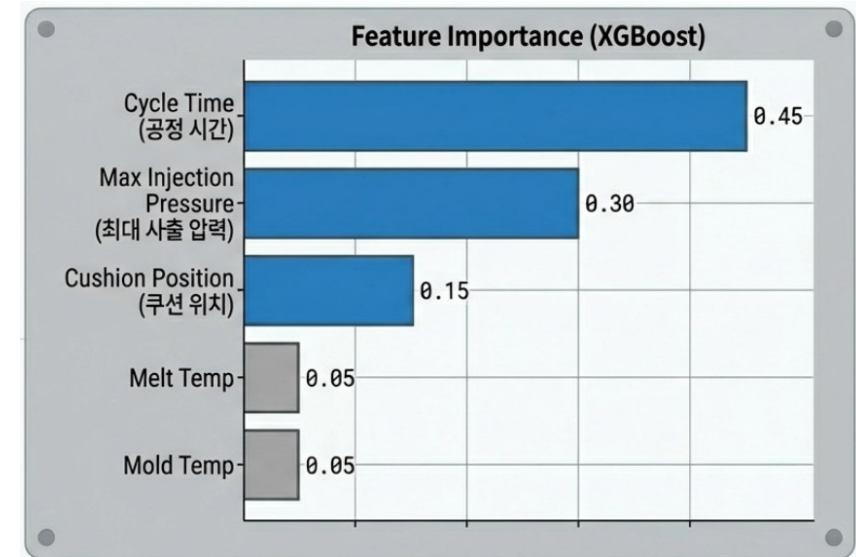
- 모델이 예측 값을 출력할 때 각 변수가 미치는 영향을 정량화한 것
- 변수 중요도를 측정하기 위해서는 모델링 (모델 학습)을 선행해야 됨

- **모델링**

- 모델: XGBoost 또는 Random Forest 추천
- 결과: 모델 정확도 98% 달성

- **변수 중요도 Top 3:**

- 공정 시간 Cycle Time: 시간이 너무 짧거나 길면 100% 불량
- 최대 사출 압력 Max Injection Pressure: 재료를 밀어넣는 힘의 일관성
- 쿠션 위치 Cushion Position: 사출 후 남은 재료의 양 (압력 누수 지표)



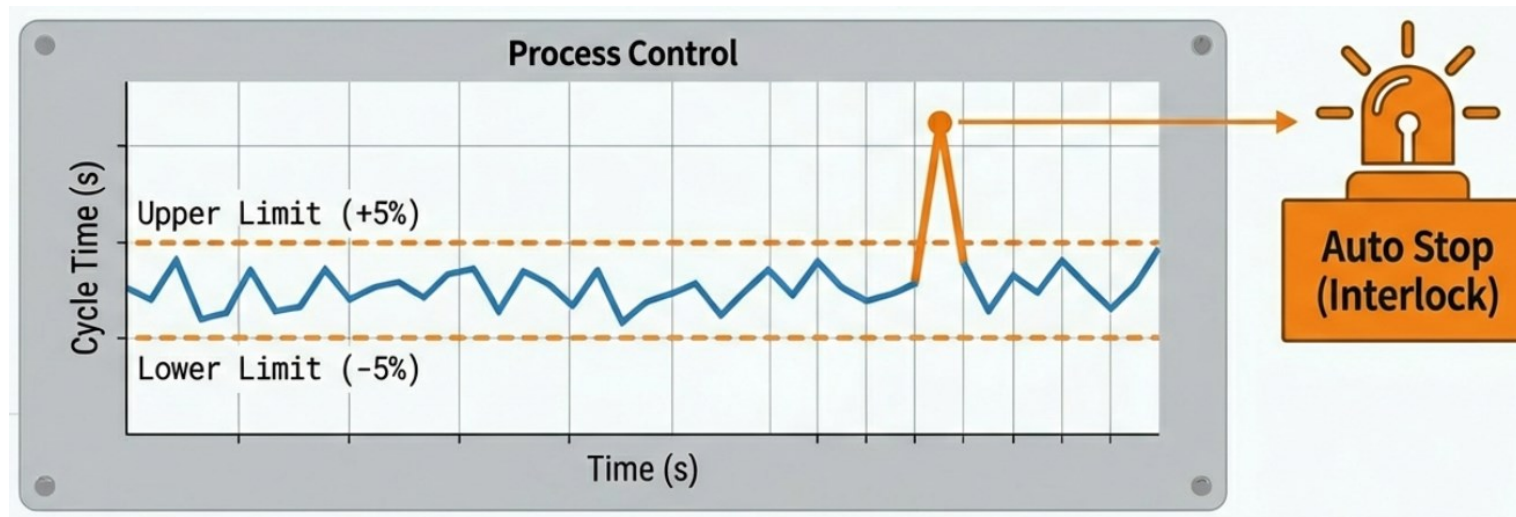
EDA: Summary

- 분석 결론

- 불량률의 주원인은 재료의 온도 문제가 아니라, 기계가 압력을 가하는 시간과 압력의 크기가 일정하지 않기 때문

- 처방

- Cycle Time이 표준 시간(예: 30초)을 $\pm 5\%$ 벗어나면 즉시 설비 정지 알람 설정
- 쿠션 위치 센서 점검 (압력 누수 확인)



References



- Connolly, T., Database Systems: A Practical Approach to Design, Implementation, and Management, 6th Edition, Pearson.
- 조재성, pandas 추가 - 계층적 인덱싱, 티스토리 ([link](#))
- VanderPlas, J., Python Data Science Handbook, GitHub ([link](#))
- Wang, J., Python Pandas DataFrame Join, Merge, and Concatenate, towards data science ([link](#))
- tiefengeist, Relational Algebra: the Underpinnings of SQL, Medium ([link](#))
- 김정인, 파이썬 데이터 사이언스 핸드북, 위키북스, 2017.



수고하셨습니다.